

Trie

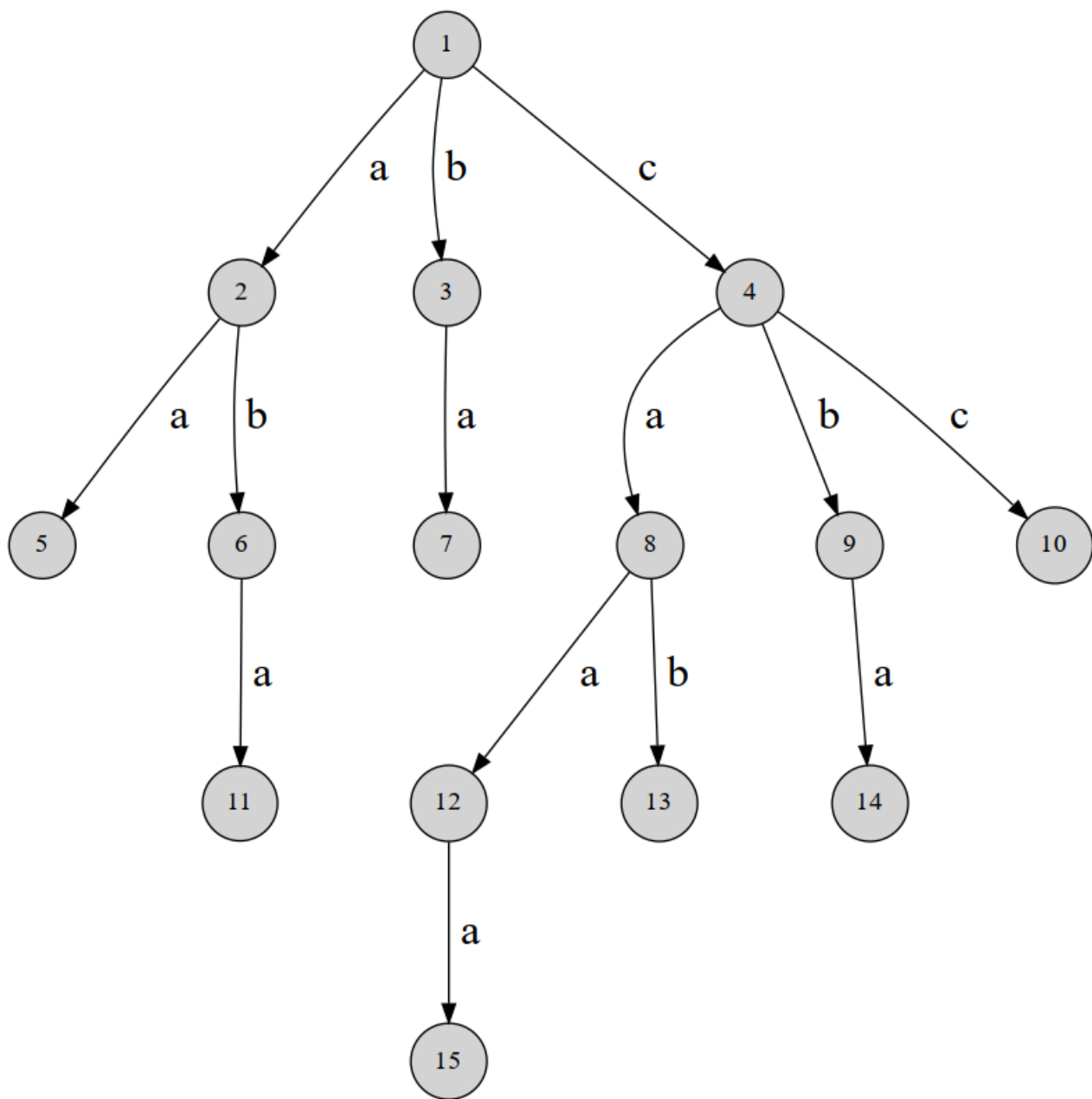
字典树 (Trie)

定义

字典树，英文名 trie。顾名思义，就是一个像字典一样的树。

引入

先放一张图：



Trie

可以发现，这棵字典树用边来代表字母，而从根结点到树上某一结点的路径就代表了一个字符串。举个例子，表示的就是字符串 `caa`。

trie 的结构非常好懂，我们用 表示结点 的 字符指向的下一个结点，或着说是结点 代表的字符串后面添加一个字符形成的字符串的结点。（的取值范围和字符集大小有关，不一定是。）

有时需要标记插入进 trie 的是哪些字符串，每次插入完成时在这个字符串所代表的节点处打上标记即可。

实现

放一个结构体封装的模板：



C++PythonJava

```
1 struct trie {
2     int nex[100000][26], cnt;
3     bool exist[100000]; // 该结点结尾的字符串是否存在
4
5     void insert(char *s, int l) { // 插入字符串
6         int p = 0;
7         for (int i = 0; i < l; i++) {
8             int c = s[i] - 'a';
9             if (!nex[p][c]) nex[p][c] = ++cnt; // 如果没有，就添加结点
10            p = nex[p][c];
11        }
12        exist[p] = true;
13    }
14
15    bool find(char *s, int l) { // 查找字符串
16        int p = 0;
17        for (int i = 0; i < l; i++) {
18            int c = s[i] - 'a';
19            if (!nex[p][c]) return 0;
20            p = nex[p][c];
21        }
22        return exist[p];
23    }
24};
```

```
1 class trie:
2     def __init__(self):
3         self.nex = [[0 for i in range(26)] for j in range(100000)]
4         self.cnt = 0
5         self.exist = [False] * 100000 # 该结点结尾的字符串是否存在
6
7     def insert(self, s): # 插入字符串
8         p = 0
9         for i in s:
10            c = ord(i) - ord("a")
11            if not self.nex[p][c]:
12                self.cnt += 1
13                self.nex[p][c] = self.cnt # 如果没有，就添加结点
14            p = self.nex[p][c]
15        self.exist[p] = True
16
17    def find(self, s): # 查找字符串
18        p = 0
19        for i in s:
20            c = ord(i) - ord("a")
21            if not self.nex[p][c]:
22                return False
23            p = self.nex[p][c]
24        return self.exist[p]
```

```

1 public class Trie {
2
3     int[][] tree = new int[10000][26];
4     int cnt = 0;
5     boolean[] end = new boolean[10000];
6
7     public void insert(String word) {
8         int p = 0;
9         char[] chars = word.toCharArray();
10        for (int i = 0; i < chars.length; i++) {
11            int c = chars[i] - 'a';
12            if (tree[p][c] == 0) {
13                tree[p][c] = ++cnt;
14            }
15            p = tree[p][c];
16        }
17        end[p] = true;
18    }
19
20    public boolean find(String word) {
21        int p = 0;
22        char[] chars = word.toCharArray();
23        for (int i = 0; i < chars.length; i++) {
24            int c = chars[i] - 'a';
25            if (tree[p][c] == 0) {
26                return false;
27            }
28            p = tree[p][c];
29        }
30        return end[p];
31    }
32 }

```

应用

检索字符串

字典树最基础的应用——查找一个字符串是否在「字典」中出现过。

▼ [于是他错误的点名开始了](#)

给你一个名字串，然后进行 m 次点名，每次你需要回答「名字不存在」、「第一次点到这个名字」、「已经点过这个名字」之一。

n 个名字，所有字符串长度不超过 10。

- 题解
- 参考代码

AC 自动机

trie 是 [AC 自动机](#) 的一部分。

维护异或极值

将数的二进制表示看做一个字符串，就可以建出字符集为 $\{0,1\}$ 的 trie 树。

▼ [BZOJ1954 最长异或路径](#)

给你一棵带边权的树，求使得从根到叶子的路径上的边权异或和最大，输出这个最大值。这里的异或和指的是所有边权的异或。

或。

点数不超过，边权在 内。

- 题解
- 参考代码

维护异或和

01-trie 是指字符集为 的 trie。01-trie 可以用来维护一些数字的异或和，支持修改（删除 + 重新插入），和全局加一（即：让其所维护所有数值递增 1，本质上是一种特殊的修改操作）。

如果要维护异或和，需要按值从低位到高位建立 trie。

一个约定：文中说当前节点 往上 指当前节点到根这条路径，当前节点 往下 指当前结点的子树。

插入 & 删除

如果要维护异或和，我们 只需要 知道某一位上 0 和 1 个数的 奇偶性 即可，也就是对于数字 1 来说，当且仅当这一位上数字 1 的个数为奇数时，这一位上的数字才是 1，请时刻记住这段文字：如果只是维护异或和，我们只需要知道某一位上 1 的数量即可，而不需要知道 trie 到底维护了哪些数字。

对于每一个节点，我们需要记录以下三个量：

- `ch[o][0/1]` 指节点 `o` 的两个儿子，`ch[o][0]` 指下一位是 0，同理 `ch[o][1]` 指下一位是 1。
- `w[o]` 指节点 `o` 到其父亲节点这条边上数值的数量（权值）。每插入一个数字 `x`，`x` 二进制拆分后在 trie 上 路径的权值都会 +1。
- `xorv[o]` 指以 `o` 为根的子树维护的异或和。

具体维护结点的代码如下所示。

```
1 void maintain(int o) {
2     w[o] = xorv[o] = 0;
3     if (ch[o][0]) {
4         w[o] += w[ch[o][0]];
5         xorv[o] ^= xorv[ch[o][0]] << 1;
6     }
7     if (ch[o][1]) {
8         w[o] += w[ch[o][1]];
9         xorv[o] ^= (xorv[ch[o][1]] << 1) | (w[ch[o][1]] & 1);
10    }
11    // w[o] = w[o] & 1;
12    // 只需知道奇偶性即可，不需要具体的值。当然这句话删掉也可以，因为上文就只利用了他的奇偶性。
13 }
```

插入和删除的代码非常相似。

需要注意的地方就是：

- 这里的 `MAXH` 指 trie 的深度，也就是强制让每一个叶子节点到根的距离为 `MAXH`。对于一些比较小的值，可能有时候不需要建立这么深（例如：如果插入数字 4，分解成二进制后为 100，从根开始插入 001 这三位即可），但是我们强制插入 `MAXH` 位。这样做的目的是为了便于全局 +1 时处理进位。例如：如果原数字是 3（11），递增之后变成 4（100），如果当初插入 3 时只插入了 2 位，那这里的进位就没了。
- 插入和删除，只需要修改叶子节点的 `w[]` 即可，在回溯的过程中一路维护即可。

▼ 实现

```

1 namespace trie {
2 constexpr int MAXH = 21;
3 int ch[_ * (MAXH + 1)][2], w[_ * (MAXH + 1)], xorv[_ * (MAXH + 1)];
4 int tot = 0;
5
6 int mknode() {
7     ++tot;
8     ch[tot][1] = ch[tot][0] = w[tot] = xorv[tot] = 0;
9     return tot;
10 }
11
12 void maintain(int o) {
13     w[o] = xorv[o] = 0;
14     if (ch[o][0]) {
15         w[o] += w[ch[o][0]];
16         xorv[o] ^= xorv[ch[o][0]] << 1;
17     }
18     if (ch[o][1]) {
19         w[o] += w[ch[o][1]];
20         xorv[o] ^= (xorv[ch[o][1]] << 1) | (w[ch[o][1]] & 1);
21     }
22     w[o] = w[o] & 1;
23 }
24
25 void insert(int &o, int x, int dp) {
26     if (!o) o = mknode();
27     if (dp > MAXH) return (void) (w[o]++);
28     insert(ch[o][x & 1], x >> 1, dp + 1);
29     maintain(o);
30 }
31
32 void erase(int o, int x, int dp) {
33     if (dp > 20) return (void) (w[o]--);
34     erase(ch[o][x & 1], x >> 1, dp + 1);
35     maintain(o);
36 }
37 } // namespace trie

```

全局加一

所谓全局加一就是指，让这棵 trie 中所有的数值 +1。

形式化的讲，设 trie 中维护的数值有，全局加一后 其中维护的值应该变成

```

1 void addall(int o) {
2     swap(ch[o][0], ch[o][1]);
3     if (ch[o][0]) addall(ch[o][0]);
4     maintain(o);
5 }

```

过程

我们思考一下二进制意义下 +1 是如何操作的。

我们只需要从低位到高位开始找第一个出现的 0，把它变成 1，然后这个位置后面的 1 都变成 0 即可。

下面给出几个例子感受一下：（括号内的数字表示其对应的十进制数字）

```
1 1000(8) + 1 = 1001(9) ;
2 10011(19) + 1 = 10100(20) ;
3 11111(31) + 1 = 100000(32);
4 10101(21) + 1 = 10110(22) ;
5 10000000011111(16447) + 1 = 100000001000000(16448);
```

对应 trie 的操作，其实就是交换其左右儿子，顺着 **交换后** 的 0 边往下递归操作即可。

回顾一下 $w[o]$ 的定义： $w[o]$ 指节点 o 到其父亲节点这条边上数值的数量（权值）。

有没有感觉这个定义有点怪呢？如果在父亲结点存储到两个儿子的这条边的边权也许会更接近于习惯。但是在这里，在交换左右儿子的时候，在儿子结点存储到父亲这条边的距离，显然更加方便。

01-trie 合并

指的是将上述的两个 01-trie 进行合并，同时合并维护的信息。

可能关于合并 trie 的文章比较少，其实合并 trie 和合并线段树的思路非常相似，可以搜索「合并线段树」来学习如何合并 trie。

其实合并 trie 非常简单，就是考虑一下我们有一个 `int merge(int a, int b)` 函数，这个函数传入两个 trie 树位于同一相对位置的结点编号，然后合并完成后返回合并完成的结点编号。

过程

考虑怎么实现？

分三种情况：

- 如果 a 没有这个位置上的结点，新合并的结点就是 b
- 如果 b 没有这个位置上的结点，新合并的结点就是 a
- 如果 a, b 都存在，那就把 b 的信息合并到 a 上，新合并的结点就是 a ，然后递归操作处理 a 的左右儿子。

提示：如果需要的合并是将 a, b 合并到一棵新树上，这里可以新建结点，然后合并到这个新结点上，这里的代码实现仅仅是将 b 的信息合并到 a 上。

实现

```
1 int merge(int a, int b) {
2   if (!a) return b; // 如果 a 没有这个位置上的结点，返回 b
3   if (!b) return a; // 如果 b 没有这个位置上的结点，返回 a
4   /*
5     如果 `a`, `b` 都存在，
6     那就把 `b` 的信息合并到 `a` 上。
7   */
8   w[a] = w[a] + w[b];
9   xorv[a] ^= xorv[b];
10  /* 不要使用 maintain(),
11     maintain() 是合并a的两个儿子的信息
12     而这里需要 a b 两个节点进行信息合并
13   */
14  ch[a][0] = merge(ch[a][0], ch[b][0]);
15  ch[a][1] = merge(ch[a][1], ch[b][1]);
16  return a;
17 }
```

其实 trie 都可以合并，换句话说，trie 合并不仅仅限于 01-trie。

▼ [【luogu-P6018】](#) [【Ynoi2010】 Fusion tree](#)

给你一棵 n 个结点的树，每个结点有权值。次操作。需要支持以下操作。

- 将树上与一个节点 u 距离为 k 的节点上的权值 v 。这里树上两点间的距离定义为从一点出发到另外一点的最短路径上边的条数。
- 在一个节点 u 上的权值 v 。
- 询问树上与一个节点 u 距离为 k 的所有节点上的权值的异或和。对于 n 的数据，满足 $n \leq 10^5$ ， $k \leq 10^5$ ， $v \leq 10^9$ 。保证任意时刻每个节点的权值非负。

► 题解

► 参考代码

▼ [【luogu-P6623】](#) [【省选联考 2020 A 卷】 树](#)

给定一棵 n 个结点的有根树，结点从 1 开始编号，根结点为 1 号结点，每个结点有一个正整数权值 w_i 。设 u 号结点的子树内（包含 u 自身）的所有结点编号为 S_u ，定义 u 的价值为：

其中 $l(u, v)$ 表示树上 u 号结点与 v 号结点间唯一简单路径所包含的边数， $\oplus$ 表示异或运算。请你求出 $\sum_{i=1}^n w_i \times f(i)$ 的结果。

► 题解

► 参考代码

可持久化字典树

参见 [可持久化字典树](#)。

本页面最近更新：2024/12/11 19:27:26，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [aofall](#), [CCXXXI](#), [Chrogeek](#), [Clouder0](#), [Enter-tainer](#), [flylai](#), [Henry-ZHR](#), [iamSmally](#), [ImpleLee](#), [lr1d](#), [kenlig](#), [Konano](#), [ksyx](#), [lingfunny](#), [Menci](#), [ouuan](#), [shawlleyw](#), [ShuYuMo2003](#), [sshwy](#), [Tiphereth-A](#), [weroicp](#), [Xeonacid](#), [Xiaoxiong-Liu](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用